AWS CloudFormation - Service Overview

Introduction

AWS CloudFormation is a service that helps you model and set up your AWS resources so you can spend less time managing those resources and more time focusing on your applications. You create a template that describes the AWS resources you want, and CloudFormation takes care of provisioning and configuring those resources for you, in the correct dependency order.

Templates

A CloudFormation template is a JSON or YAML formatted text file describing the desired resources and their configuration. Templates commonly include sections for Parameters (input values supplied at stack creation), Resources (the AWS resources to create, the only required section), Outputs (values to export after stack creation, such as a resource's generated ID or ARN), and Mappings (static lookup tables).

Stacks

A stack is a single unit of resources managed together as a group, created and updated by applying a template. When you delete a stack, CloudFormation automatically deletes all the resources associated with it (unless a resource has deletion protection or a retain policy explicitly configured), which helps avoid orphaned resources.

Change Sets

Before applying an update to an existing stack, you can generate a change set, which shows a preview of the changes CloudFormation will make (which resources will be added, modified, or replaced) without actually executing them. This allows you to review the potential impact, including any resource replacements, which typically involve downtime.

Drift Detection

Drift detection identifies resources that have been modified outside of CloudFormation, for example by a manual change made directly in the AWS console, allowing you to detect and reconcile configuration that has diverged from what the template describes.

Nested Stacks and Modules

For complex infrastructure, CloudFormation supports nested stacks, where one template references and creates another stack as a resource, promoting reuse of common infrastructure patterns (such as a standard VPC layout) across multiple higher-level stacks.

Relationship to Terraform

CloudFormation is AWS's native infrastructure-as-code service, while Terraform (by HashiCorp) is a popular multi-cloud alternative. Both follow a declarative model, where you describe the desired end state and the tool determines the necessary actions. Terraform maintains its own state file to track managed resources, whereas CloudFormation maintains state internally within the stack itself, without a separate state file to manage.

Intrinsic Functions

CloudFormation templates support intrinsic functions such as `Fn::GetAtt` (retrieve an attribute of a resource, like an ALB's DNS name), `Fn::Sub` (string substitution), and `Ref` (reference a parameter or resource), enabling dynamic values within an otherwise static template.

StackSets

AWS CloudFormation StackSets extends this functionality by enabling you to create, update, or delete stacks across multiple accounts and regions with a single operation, useful for deploying a consistent baseline (such as security guardrails) across an entire AWS Organization.

Rollback Behavior

If a resource fails to create or update during a stack operation, CloudFormation by default automatically rolls back the entire stack to its last known good state, undoing any resources that were successfully created or modified earlier in that same operation. This all-or-nothing behavior prevents a stack from being left in a partially applied, inconsistent state, though it can make debugging the root cause of a failure harder since the failing resource's evidence may be rolled back before you can inspect it; disabling rollback temporarily during development can help with troubleshooting.

Relationship to AWS CDK

The AWS Cloud Development Kit (CDK) lets developers define infrastructure using familiar programming languages such as Python, TypeScript, or Java, rather than writing raw JSON or YAML. Under the hood, the CDK synthesizes a standard CloudFormation template from your code, meaning CDK does not replace CloudFormation but instead generates it, giving you the expressive power of a full programming language (loops, conditionals, reusable classes) while still deploying through CloudFormation's underlying stack and change-set mechanisms.

Common Use Cases

- Reproducibly provisioning a full application environment (VPC, compute, database, IAM roles) from a single template
- Standardizing infrastructure patterns across multiple teams or environments via nested stacks
- Deploying consistent security and compliance baselines across many AWS accounts via StackSets

Best Practices

- Use change sets before every production stack update to review the blast radius of a change, especially where resource replacement (and downtime) might be triggered.
- Parameterize environment-specific values (like instance size or environment name) rather than hardcoding them, so the same template can be reused across dev/staging/prod.
- Enable termination protection on stacks containing critical, hard-to-recreate resources such as production databases.